

Aufgabe 1 (TicTacToe MLP)

Sie haben in der letzten Übung eine KI mittels Alpha-Beta-Pruning erstellt. Nun sollen Sie für das Spiel TicTacToe basierend auf bereits gelösten Spielen (mittels eines einfachen MiniMax Algorithmus) ein Multi-Layer-Perzeptron (MLP) erstellen.

Das MLP nimmt das Spielbrett, bestehend aus 9 Werten als Numpy Array wie folgt an:

```
0 1 2
3 4 5
6 7 8
```

Dabei steht bei einem Feld: 1 für X, -1 für O und 0 für noch ein nicht besetztes Feld. Das MLP gibt für jedes Feld einen Wert aus und bestimmt mit dem höchsten Wert, welchen Zug die KI als nächstes durchführt.

1. Erstellen Sie zunächst das MLP. Verwenden Sie dazu die [ReLU](#)-Aktivierungsfunktion anstatt der in der Vorlesung verwendeten [Sigmoid](#)-Funktion. Diese Funktion zeigt im Training eines KI-Modells bei den meisten Anwendungsfällen, eine deutlich schnellere Konvergenz. Verwenden Sie zwei Hidden-Layer mit je 64 Neuronen.

```
In [ ]: import numpy as np
import torch
import torch.nn as nn
import mlflow

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("TicTacToe")

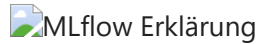
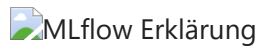
class TicTacToeMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            #
            # Ihr Netz wird hier definiert:
            #
            nn.Linear(9, ...), # Eingabe-Layer
            nn.ReLU(), # Aktivierungsfunktion
            # ... mehrere Operationen
            nn.Linear(..., 9), # Ausgabe-Layer
            #
            # ^^^^^^^^^^^^^^^^^
            #
        )

    def forward(self, x):
        return self.net(x)
```

2. `X` ist eine Liste aus Tupeln `(Board, Player)`, `y` enthält für jedes Board alle möglichen Züge als Indizes. Führen Sie einen Trainingsdurchlauf durch, bei dem alle Züge als Many-Hot-Encoding dargestellt werden. Starten Sie vor dem Training die

MLflow-UI und beobachten Sie während des Trainings den Verlauf der Verlust- bzw. Metrikkurve. Prüfen Sie, ob das Modell tatsächlich lernt, ob sich die Leistung kontinuierlich verbessert oder ob das Training ab einem bestimmten Zeitpunkt stagniert bzw. instabil wird.

Sobald MLflow läuft, sieht man wie folgt die Metriken:



MLflow ui lokal starten: `mlflow ui --host 0.0.0.0` im Terminal ausführen. Dann <http://localhost:5000/> öffnen.

MLflow in JupyterLab starten, dazu im Terminal ausführen:

```
export MLFLOW_SERVER_CORS_ALLOWED_ORIGINS="https://*.mni.thm.de:443"
export MLFLOW_SERVER_ALLOWED_HOSTS="*.mni.thm.de:*"
mlflow ui --host 0.0.0.0
```

Dann untere Zelle ausführen und auf den Link klicken.

```
In [ ]: # Adresse für MLflow in JupyterLab
import os
print(f"https://jl.mni.thm.de/user/{os.environ['JUPYTERHUB_USER']}/proxy/5000/")
```

```
In [ ]: # Hier muss nichts zur Erfüllung der Aufgabe geändert werden
def train_tictactoe(
    X, y, epochs: int = 1000, lr: float = 0.001
):
    model = TicTacToeMLP()
    loss_fn = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    with mlflow.start_run():
        for epoch in range(1000):
            logits = model(X)
            loss = loss_fn(logits, y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            mlflow.log_metric("train_loss", loss.item(), step=epoch)
            if epoch % 100 == 0:
                print("Finished epoch:", epoch)
        print("Finished training, end loss", loss.item())

    return model
```

```
In [ ]: from gen_tictactoe import build_dataset
import torch.nn.functional as F

X, y = build_dataset()
# Spieler-Info ignorieren
X = torch.tensor(
    np.array([e[0] for e in X], dtype=float),
    dtype=torch.float32
```

```

)

#
# Erstellen Sie hier ihre many hot encoding!
#
many_hot_y = ...
#
# Bis hier!
#
model = train_tictactoe(X, many_hot_y)

```

3. Was fällt Ihnen bei der Verlustkurve auf?

IHRE ERKLÄRUNG HIER

Aufgabe 2 (MNIST MLP)

Sie haben nun einen Datensatz zur Zahlenerkennung (0 bis 9) und sollen mithilfe eines MLPs die auf 28×28 Pixel großen Bilder einer Zahlenklasse zuordnen.



1. Beginnen Sie zunächst mit dem Modell. Verwenden Sie ein Hidden Layer mit 128 Knoten.

```
In [ ]: mlflow.set_experiment("MNIST")

class MNISTMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(), # Eingabelayer, Bild zu einem Array flatten
            # Ihr Modell ab hier:
            nn.Linear(28*28, 10), # Ausgabelayer
        )

    def forward(self, x):
        return self.net(x)
```

2. Nachdem Sie das Modell mit einem Hidden Layer und 128 Knoten trainiert haben, trainieren Sie das Modell mit zwei Hidden Layern (erster Layer 48 Knoten und zweiter Layer 32 Knoten). Welches Modell ist anhand der Verlustfunktion besser?

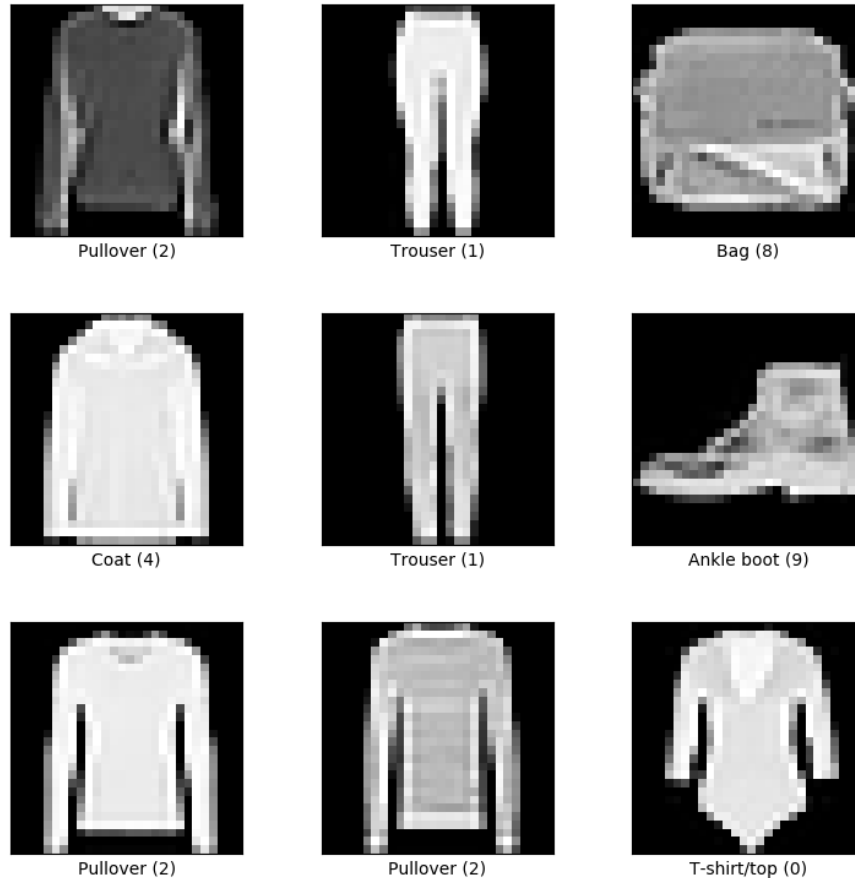
```
In [ ]: from MNIST_trainer import train_mnist
model = MNISTMLP()
# Alle Parameter, basierend auf Layers:
print(sum(p.numel() for p in model.parameters()))
train_mnist(model)
```

3. Haben Sie eine mögliche Erklärung wieso ein Modell besser ist als das andere? Was sind die Parameter eines MLP? Was passiert, wenn ein MLP zu wenig Parameter hat?

IHRE ERKLÄRUNG HIER

Aufgabe 3 (FashionMNIST MLP)

Passen Sie nun die Aufgabe 2 so an, dass Sie FashionMNIST Daten (mit `datasets.FashionMNIST`) laden können. Anders als bei MNIST versucht man hier mit 10 Klassen für Kleidungsstücke zu arbeiten. Die Bildeingaben sind jedoch immer noch 28×28 Pixel große Bilder.



1. Erstellen Sie zunächst ein MLP, mit zwei Hidden Layern (erster Hidden Layer 256 Knoten, zweiter Hidden Layer 128 Neuronen).

```
In [ ]: # Ihr MLP Modell
mlflow.set_experiment("FashionMNIST")

class FashionMNISTMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            # ...
        )

    def forward(self, x):
        return self.net(x)
```

2. Trainieren Sie für mindestens 4 Epochen. Nachdem Sie das Modell trainiert haben, trainieren Sie ein weiteres Modell mit 3 Hidden Layern (384 Knoten, 256 Knoten und 128 Knoten).

```
In [ ]: # Schreiben Sie hier Code um Ihr Modell zu trainieren
# Schauen Sie sich MNIST_trainer.py an:
# - Ändern Sie datasets.MNIST zu datasets.FashionMNIST
```

3. Fällt Ihnen etwas bei der Verlustkurve der beiden Modelle auf? Wie sieht der Verlust beim Validieren aus? Ist größer immer besser?

IHRE ERKLÄRUNG HIER

Aufgabe 4 (MLP Betrachtung)

Testen Sie nun exemplarisch anhand des FashionMNIST oder MNIST Datensatzes, ob tiefere (5 Hidden Layer) oder breitere Netze (max. 1 Hidden Layer) besser sind. Versuchen Sie ihre Parameteranzahl dabei gleich zu halten um eine vergleichbare Metrik zu haben. Testen Sie auch einmal ein kleines tiefes Netz (min 10 Layer, weniger Parameter).

Was sind Ihre Erkenntnisse?

IHRE ERKLÄRUNG HIER